

Telemetry-Software Integration

Technical Guidelines

Version 1.0 – September 15th 2026

University of Puerto Rico, Mayagüez Campus (UPRM)

Prepared by: **Javier E. Veras Seda, Sebastian M. Pagan Olivera**

Table of Contents

ABSTRACT	3
REQUIREMENTS & OBJECTIVES	3
2.1 DATA ACQUISITION	3
2.2 SIGNAL QUALITY & ACCURACY	3
2.3 TELEMETRY DATA ON GUI	3
2.4 TROUBLESHOOTING & DIAGNOSTICS	4
2.5 EXPORTING TELEMETRY DATA	4
PRELIMINARY CONCEPT DESIGN	5
3.1 QUANTITY OF SIGNALS	5
3.2 WARNINGS DISPLAYED	6
3.3 GUI VISUAL DESIGN	6
APPENDIX	7
A. DATA IDENTIFIERS	7
B. FAULT CODES	7

Abstract

This document defines the telemetry requirements to be implemented by **the Software Division**. It specifies the telemetry signals to be transmitted to the Jetson Nano, their acquisition methods, and formatting guidelines. It also defines the placement and presentation of telemetry data within the rover's GUI, including real-time readings, statistics, trouble codes, warnings, and other system-status information. Preliminary design requirements cover the number and type of signals to be monitored, methods for reading specific signals, GUI functionality, and troubleshooting considerations. The document serves as a reference for implementing the software side of the rover's telemetry system.

Requirements & Objectives

2.1 Data Acquisition

The telemetry serial link shall operate at a **115200 baudrate**. The Microcontroller shall initialize its USB serial interface using [*Serial.begin(115200)*], while the Jetson Nano shall configure the corresponding Linux serial device to the same communication rate.

Telemetry data shall be transmitted through the serial interface using a structured, ASCII-encoded frame with the following format:

XXX=##.##,XXX=##.##\n

Each telemetry variable shall consist of a predefined **variable identifier**, followed by the assignment character (=) and its corresponding numerical value. Individual variables shall be separated by a comma (,), while each complete telemetry frame shall be terminated by a newline character (\n).

For a more detailed look at the specific signals to be read, see **Section 3.1**.

2.2 Signal Quality & Accuracy

As previously specified, each telemetry signal shall be represented with a minimum numerical resolution of **two decimal places**. All telemetry values shall be appropriately scaled, converted, and validated for measurement accuracy prior to serial transmission. Signal acquisition, calibration, conversion, and validation shall remain the responsibility of the **Power & Electrical Systems Division**.

Complete telemetry frames shall be transmitted to the Jetson Nano at a fixed rate of **4 frames per second (4 Hz)**, corresponding to a transmission interval of **250 ms per frame**.

2.3 Telemetry Data on GUI

Each telemetry signal shall be individually inspectable through the GUI. For every received signal, the interface shall display its **current value**, **minimum recorded value**, **maximum recorded value**, and **average value since system startup**. Each signal shall additionally have an associated **time-series graph** displaying its historical value as a function of elapsed time, using the appropriate engineering unit (V, A, °C, etc.).

The displayed current value and corresponding time-series graph shall be updated at the specified telemetry frame rate of **4 Hz**, resulting in one GUI data update every **250 ms**. Upon reception and successful parsing of each telemetry frame, the

GUI shall append the new sample to the corresponding signal history and refresh the displayed current, minimum, maximum, and average values.

2.4 Troubleshooting & Diagnostics

The lowercase identifier **f** shall designate a given field as a **fault variable**, while **XX** shall represent a unique fault index used exclusively to distinguish individual fault entries within the telemetry frame.

The four-digit numerical value shall encode the origin and type of the detected fault using a predefined hierarchical structure. The **first digit** shall identify the rover subsystem in which the fault originated. The **second digit** shall identify the specific telemetry signal within that subsystem responsible for generating the fault. The **remaining two digits** shall identify the specific type of fault detected for that signal.

The general fault-code structure shall therefore be defined as:

fXX=SDFF

where the individual digits represent:

- **S**— Subsystem identifier
- **D** – Signal identifier within the specified subsystem
- **FF** – Fault-type identifier
- **XX** – Unique fault index within the transmitted telemetry frame

For example, the transmitted field:

f02=3402

shall identify fault index 02, originating from **subsystem 3**, associated with **signal 4** within that subsystem, and corresponding to **fault type 02**.

The Software Division shall decode received fault codes according to this structure and use the subsystem and signal identifiers to associate each fault with its corresponding GUI telemetry element. The fault-type identifier shall subsequently be used to retrieve the appropriate fault description.

For a complete definition of subsystem identifiers, signal identifiers, fault-type identifiers, and their corresponding descriptions, refer to **Appendix A**.

2.5 Exporting Telemetry Data

The Software Division shall implement a feature allowing all data collected during the current operating session to be exported as an **.xlsx workbook**. Session telemetry shall be stored continuously in memory or an appropriate local data structure as frames are received, allowing the complete dataset to be written to the workbook upon an export request.

The exported workbook shall contain:

- **Telemetry Data:** All signal readings organized against elapsed time since system startup.

- **Signal Statistics:** Minimum, maximum, and average values for each telemetry signal.
- **Fault Log:** All registered fault events, including fault code, corresponding message, subsystem, originating subsystem signal, and session timestamp.

The Software Division shall be responsible for implementing the .xlsx generation process and determining the workbook structure and formatting. Export operations shall not interrupt the established **4 Hz telemetry acquisition process**.

Preliminary Design Specifics

3.1 Quantity of Signals

The telemetry system shall transmit **17 independent real-time signals** from the Microcontroller to the Jetson Nano. Only the instantaneous signal readings shall be transmitted.

The Software Division shall be responsible for receiving, parsing, and storing the real-time signal data and using the received samples to calculate the statistical values for each signal during the current operating session. Each signal is also expected to be represented in accordance to Section 2.3.

TABLE 1. TELEMETRY SIGNALS & IDENTIFIERS

Identifier	Signal	Unit
12V	12 V Bus Voltage	V
12C	12 V Bus Current	A
05V	5 V Bus Voltage	V
05C	5 V Bus Current	A
24V	24 V Bus Voltage	V
24C	24 V Bus Current	A
BTP	Battery Temperature	°F
BC1	Battery Cell 1 Voltage	V
BC2	Battery Cell 2 Voltage	V
BC3	Battery Cell 3 Voltage	V
BC4	Battery Cell 4 Voltage	V
BVS	Battery Cell Sum	V

FRC	Front-Right Wheel Current	A
FLC	Front-Left Wheel Current	A
BRC	Back-Right Wheel Current	A
BLC	Back-Left Wheel Current	A
WCS	Wheel Current Sum	A

For each real-time signal, the Software Division shall generate and maintain:

Real-Time Value | Minimum | Maximum | Average

Restricting transmission to real-time readings reduces the amount of data contained within each serial frame and therefore **reduces the communication load between the microcontroller and Jetson Nano**.

The total number of telemetry signals relayed within each serial frame shall remain dynamic to accommodate the transmission of fault data. The system shall support the transmission of 17 real-time telemetry signals and a maximum of 20 fault codes per frame. When fault data is present, fault variables shall be incorporated into the transmitted frame alongside the required telemetry data. The frame structure shall therefore dynamically adjust according to the number of active faults, up to the specified maximum of 20 fault-code entries per frame, and a **total maximum of 37 variable readings**.

3.2 Warnings Displayed

Code Repository:

[Insert links to repositories or cloud storage where the code and its iteration are housed. If code segment is very short it can be included as an image with the same titles directions as seen in the previous section.]

Programmable Logic:

[Explain the logic behind the control aspect or code the developed sub-system uses. It can be a surface level explanation in case of extensive coding]

3.3 GUI Visual Design

Code Repository:

[Insert links to repositories or cloud storage where the code and its iteration are housed. If code segment is very short it can be included as an image with the same titles directions as seen in the previous section.]

Programmable Logic:

[Explain the logic behind the control aspect or code the developed sub-system uses. It can be a surface level explanation in case of extensive coding]

Appendix

A. DATA IDENTIFIERS

B. FAULT CODES